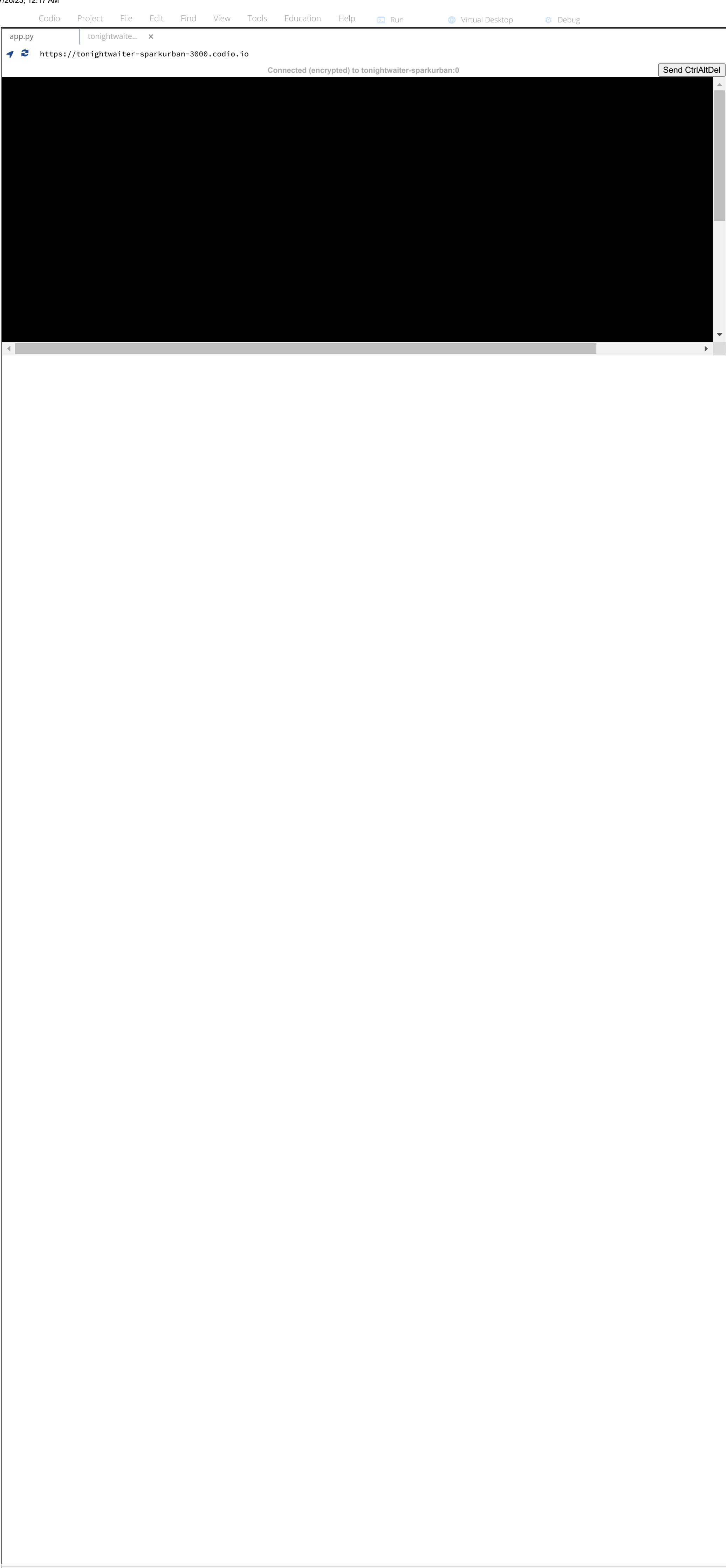




```
(base) codio@tonightwaite-sparkurban:~/workspace/app$ conda activate gui
(gui) codio@tonightwaite-sparkurban:~/workspace/app$ python app.py
[INFO ] [Logger]      Record log in /home/codio/.kivy/logs/kivy_23-07-25_1.txt
[INFO ] [Kivy]        v2.2.1
[INFO ] [Kivy]        Installed at "/home/codio/anaconda3/envs/gui/lib/python3.10/site-packages/kivy/_init__.py"
[INFO ] [Python]      v3.10.12 | packaged by conda-forge | (main, Jun 23 2023, 22:48:32) [GCC 12.3.0]
[INFO ] [Python]      Interpreter at "/home/codio/anaconda3/envs/gui/bin/python"
[INFO ] [Logger]      Purge log fired. Processing...
[INFO ] [Logger]      Purge finished!
[INFO ] [Factory]     158 symbols loaded
[INFO ] [Image]      Providers: img_tex, img_dds, img_sdl2, img_pil (img_ffpyplayer ignored)
[INFO ] [Text]       Provider: sdl2
[ERROR ] [Input]      MDev is not supported by your version of linux
Traceback (most recent call last):
  File "/home/codio/anaconda3/envs/gui/lib/python3.10/site-packages/kivy/input/providers/_init__.py", line 41, in <module>
    import kivy.input.providers.mtdev
  File "/home/codio/anaconda3/envs/gui/lib/python3.10/site-packages/kivy/input/providers/mtdev.py", line 94, in <module>
    from kivy.lib.mtdev import Device, \
  File "/home/codio/anaconda3/envs/gui/lib/python3.10/site-packages/kivy/lib/mtdev.py", line 30, in <module>
    libmtdev = cdll.LoadLibrary('libmtdev.so.1')
  File "/home/codio/anaconda3/envs/gui/lib/python3.10/ctypes/_init__.py", line 452, in LoadLibrary
    return self._dlltype(name)
  File "/home/codio/anaconda3/envs/gui/lib/python3.10/ctypes/_init__.py", line 374, in _init_
    self._handle = _dlopen(self._name, mode)
OSError: libmtdev.so.1: cannot open shared object file: No such file or directory
[INFO ] [Window]      Provider: sdl2
[INFO ] [GL]         Using the "OpenGL" graphics system
```



## Kivy Widgets

### Creating Widgets

Activate the `gui` virtual environment.

```
conda activate gui
```

Before we get started with widgets, let's address the window. It should be smaller and a different color than black. First, import `Window` from `kivy.core.window`.

```
from kivy.core.window import Window
```

Set `Window.clearcolor` to the color we want for the background. We are using hexadecimal (you can also use RGBA) for our colors, so they should be represented as a string. Then set the size of the window using a tuple. Ours will be 150 pixels wide and 300 pixels tall.

```
# define the window
Window.clearcolor = "#34495e"
Window.size = (150, 300)
self.window = Window()

# define the widgets
title = Label(text="Click a Button")
```

Content is added to a window through the use of widgets. There are a variety of widgets available to you. However, we are only going to use some labels and buttons.

A label widget is used to display text on a Kivy application. Create the variable `title` as an instance of a `Label` class. The `text` parameter controls the text that appears in the label. We want our label to say "Click a Button".

```
# define the widgets
title = Label(text="Click a Button")
```

Next, create another label, but we want to be able to change the value it displays. Create `self.count` and set its value to zero. Then create the `self.count_label`. The `text` parameter accepts only strings, so typecast `self.count` as a string. Notice that we used `self` in front of the button widget names. We do this because we need to reference this widget outside of the `build` method. We do not interact with or change `title`, so it does not need `self` as a prefix.

```
# define the widgets
title = Label(text="Click a Button")
self.count = 0
self.count_label = Label(text=str(self.count))
self.add_button = Button(text="+")
self.subtract_button = Button(text="-")
```

The last widgets we are going to create are the `add_button` and `subtract_button` widgets. They should be instances of the `Button` class and the text in the button should be a plus sign and minus sign respectively.

```
# define the widgets
title = Label(text="Click a Button")
self.count = 0
self.count_label = Label(text=str(self.count))
self.add_button = Button(text="+")
self.subtract_button = Button(text="-")
```

### Adding the Widgets

You will not see the widgets in the Kivy application until you add them to the window. **Important**, the order in which you add the widgets in your code determines the order in which you see them in the application. Since our application has only one column, the widgets will appear one atop another.

Use the `add_widget()` method on `self.window` to add a widget to the GUI. Pass the widget you want to the aforementioned method. We want to first see the title, followed by the add button, the counter, and finally the subtract button.

```
# add widgets
self.window.add_widget(title)
self.window.add_widget(self.add_button)
self.window.add_widget(self.count_label)
self.window.add_widget(self.subtract_button)
```

Run your application and use the link below to see the output. The application is quite boring and plain. If you click the buttons you should see a visual cue that the application registered a click. However, the clicks do not do anything yet.

```
python app.py
```

Preview the GUI

**Try this variation:**

- Change the order of widgets to be in the reverse order.

**Solution**

Run your application with the command below and click the link to see the output.

```
python app.py
```

Preview the GUI

Lastly, deactivate the `gui` virtual environment.

```
conda deactivate
```

#### ▼ Code

Your code should look like this:

```
from kivy.app import App
from kivy.uix.gridlayout import GridLayout
from kivy.uix.label import Label
from kivy.uix.button import Button
from kivy.core.window import Window

class ClickButtons(App):
    def build(self):
        # define the window
        Window.clearcolor = "#34495e"
        Window.size = (150, 300)
        self.window = Window()

        # define the widgets
        title = Label(text="Click a Button")
        self.count = 0
        self.count_label = Label(text=str(self.count))
        self.add_button = Button(text="+")
        self.subtract_button = Button(text="-")

        # add widgets
        self.window.add_widget(title)
        self.window.add_widget(self.add_button)
        self.window.add_widget(self.count_label)
        self.window.add_widget(self.subtract_button)

        return self.window

if __name__ == "__main__":
    ClickButtons().run()
```

#### Reading Question

Which method is used to place a widget on a Kivy layout?

- ☒ `add_widget()`
- ☐ `widget()`
- ☐ `attach_widget()`
- ☐ `bind_widget()`

The `add_widget()` method is used to attach a widget to a Kivy layout. In sample GUI application, adding the `title` widget to the layout looks like this:

```
self.window.add_widget(title)
```

Check It

Next